Internet of things (Section B)⇒

Problem Statement →

based on the input from the Part A , we have desided to work on human activty recognisation(HAR) in their active times

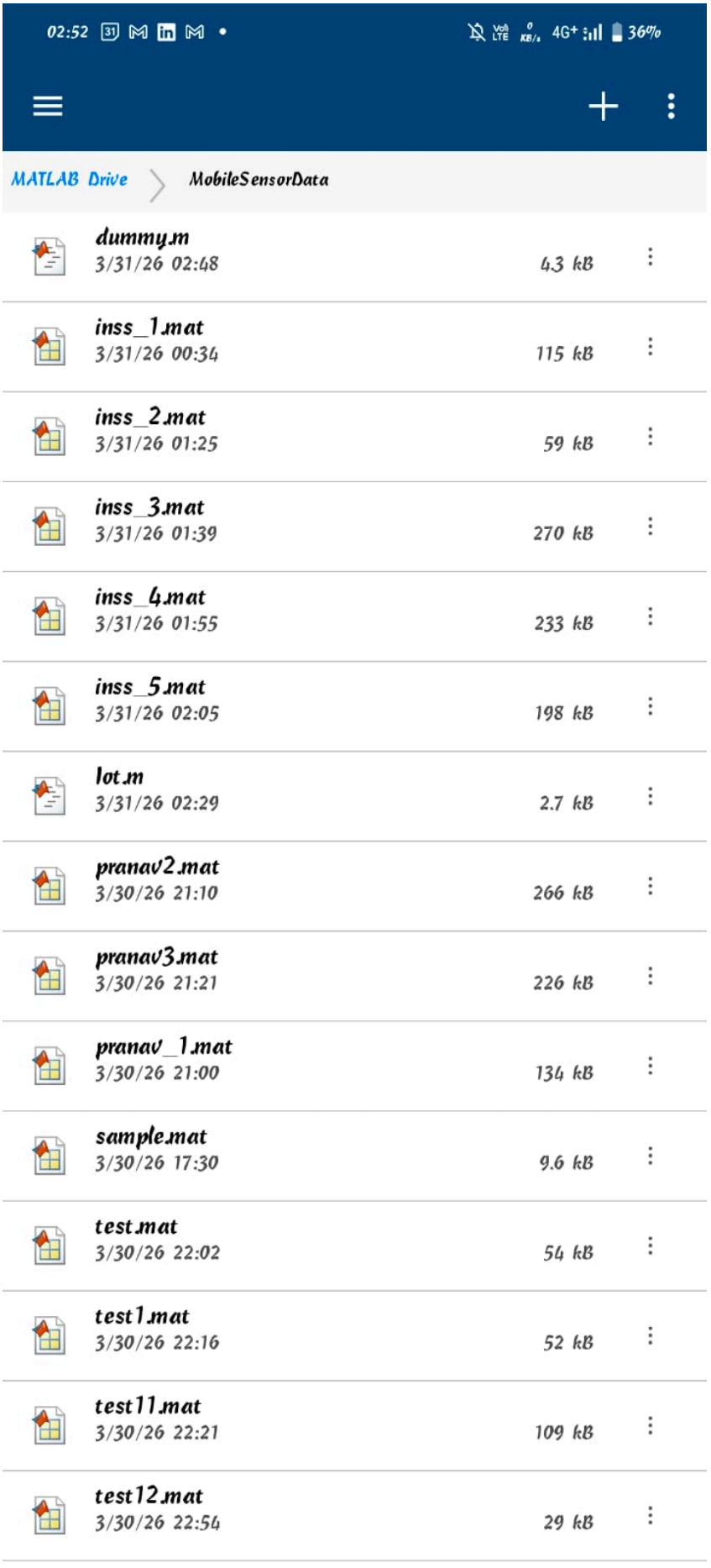
1. standing
2. sitting
3. walking
4. running

And we want to conclude a final activty score with features like →

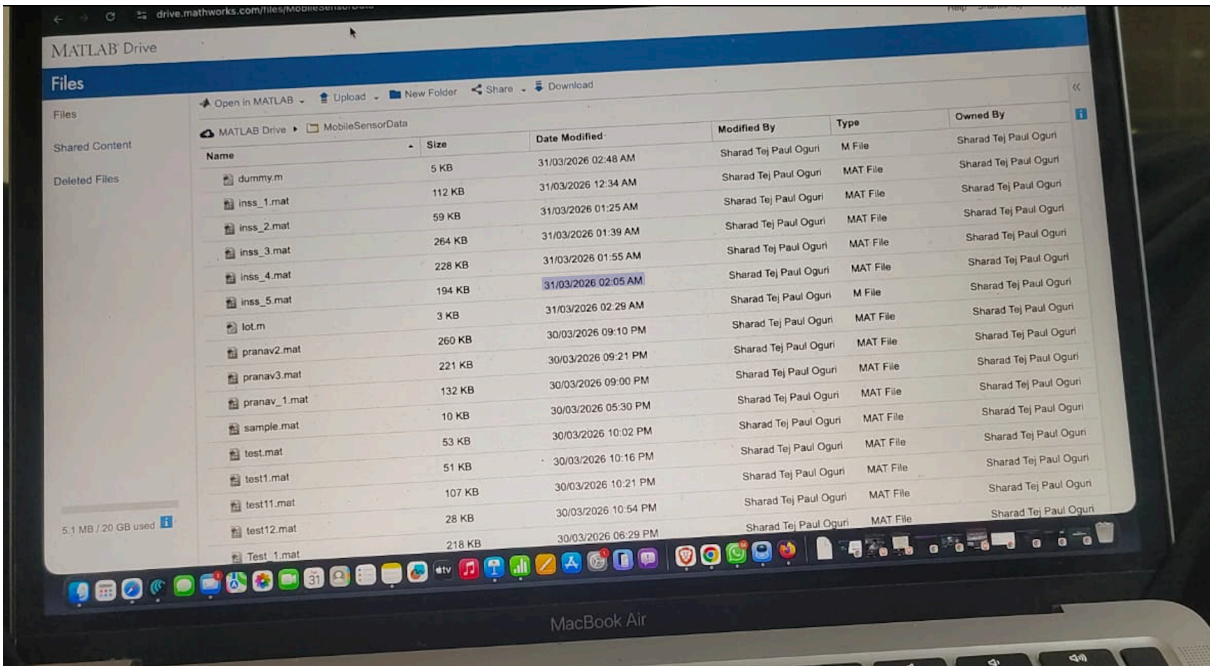
1. Mobility Score
2. Activity Intensity Score
3. Direction Change Score
4. Cummulative activity score

So the ultimate goal of the project is to support the people who want to get into fitness and are new to regular intense excerise , the current products in the market are little unmotive because they give out the results directly and they discourage people , so to support them we came up with new features which would give their active scores even if they perform tasks like sitting and standing , so that they feel engaged.

data collection using matlab mobile app→


02:52
36%
MATLAB Drive
MobileSensorData
dummy.m
3/31/26 02:48
4.3 kB
inss_1.mat
3/31/26 00:34
115 kB
inss_2.mat
3/31/26 01:25
59 kB
inss_3.mat
3/31/26 01:39
270 kB
inss_4.mat
3/31/26 01:55
233 kB
inss_5.mat
3/31/26 02:05
198 kB
lot.m
3/31/26 02:29
2.7 kB
pranav2.mat
3/30/26 21:10
266 kB
pranav3.mat
3/30/26 21:21
226 kB
pranav_1.mat
3/30/26 21:00
134 kB
sample.mat
3/30/26 17:30
9.6 kB
test.mat
3/30/26 22:02
54 kB
test1.mat
3/30/26 22:16
52 kB
test11.mat
3/30/26 22:21
109 kB
test12.mat
3/30/26 22:54
29 kB

accessing files through Matlab cloud →



`Evidence of datapoints →`

inss_5.mat

inss_4.mat

inss_3.mat

inss_2.mat

inss_1.mat

Data Sets →

we have collected the data points using the matlab sensors and the sensors used are

1. orientation sensor
2. acceleration sensor

and each data set is of 3 minute size with (sitting + standing + running + walking)

Graphs of the data points →

Task 1 → first code will help us to extract the raw data and plot the graphs with acceleration and orientation magnitudes.

```
clc; clear; close all;
filename = 'inss_4.mat';
if ~isfile(filename)
    error('File not found!');
end

data = load(filename);
vars = fieldnames(data);

disp("Variables found:");
disp(vars);

%% FILTER SETTINGS
fs = 50;
cutoff = 5;

[b,a] = butter(4, cutoff/(fs/2), 'low');
figure('Name','SENSOR DASHBOARD (NO POSITION)','Position',
[100 100 1200 800]);
plotIndex = 1;
maxPlots = 6;
```

```
for i = 1:length(vars)

    varName = vars{i};
    varData = data.(varName);

    fprintf('\nProcessing: %s\n', varName);
    if contains(lower(varName), 'pos')
        continue;
    end

    if istimetable(varData)

        tt = varData;
        time = seconds(tt.Properties.RowTimes - tt.Properti
es.RowTimes(1));

        if width(tt) >= 3

            raw = tt{:,1:3};

            if plotIndex <= maxPlots

                subplot(3,2,plotIndex);

                plot(time, raw(:,1), 'r'); hold on;
                plot(time, raw(:,2), 'g');
                plot(time, raw(:,3), 'b');

                title([varName ' (Raw XYZ)']);
                xlabel('Time (s)');
                ylabel('Value');

                legend('X','Y','Z');
                grid on;

                plotIndex = plotIndex + 1;
            end
```

```
            raw_mag = sqrt(sum(raw.^2,2));

            mag_filtered = filtfilt(b, a, raw_mag);
            mag_filtered = sgolayfilt(mag_filtered, 3, 11);

            if plotIndex <= maxPlots

                subplot(3,2,plotIndex);

                plot(time, raw_mag, '--'); hold on;
                plot(time, mag_filtered, 'LineWidth',1.5);

                title([varName ' Magnitude (Filtered)']);
                xlabel('Time (s)');
                ylabel('Magnitude');

                legend('Raw','Filtered');
                grid on;

                plotIndex = plotIndex + 1;
            end
        end
    else
        fprintf('Skipping %s (not timetable)\n', varName);
    end
end
```

serves as the **Data Visualization and Preprocessing Dashboard** for the system. Its primary objective is to transform raw, noisy signals from your smartphone into a clean, human-readable format that can be used for further analysis .

1. Data Ingestion and Validation

The script begins by targeting a specific data file, `inss_4.mat`, which contains the measurements streamed from your smartphone.

- **File Check**: It includes an error-handling block to ensure the data file exists before attempting to process it.
- **Dynamic Variable Handling**: It uses `fieldnames` to automatically detect which sensors were recorded (e.g., Acceleration, Orientation, Angular Velocity), allowing the code to be flexible regardless of which sensors you chose for your project.
- **Coordinate Independence (Magnitude)**: Because a smartphone's X, Y, and Z axes change depending on how the user holds the phone, the code calculates the **Euclidean Norm** or Magnitude:

$$Mag = \sqrt{X^2 + Y^2 + Z^2}$$

This ensures that a "step" or "movement" is detected regardless of whether the phone is vertical or horizontal in a pocket.

2. **Butterworth Low-Pass Filter**:

- The script designs a **4th-order Butterworth filter** with a **5 Hz cutoff frequency**.
- This is critical because human movement (walking/sitting) typically occurs at low frequencies (below 5 Hz), while electronic noise and vibrations occur at higher frequencies. This filter "cleans" the signal of jitter.

Smoothing:

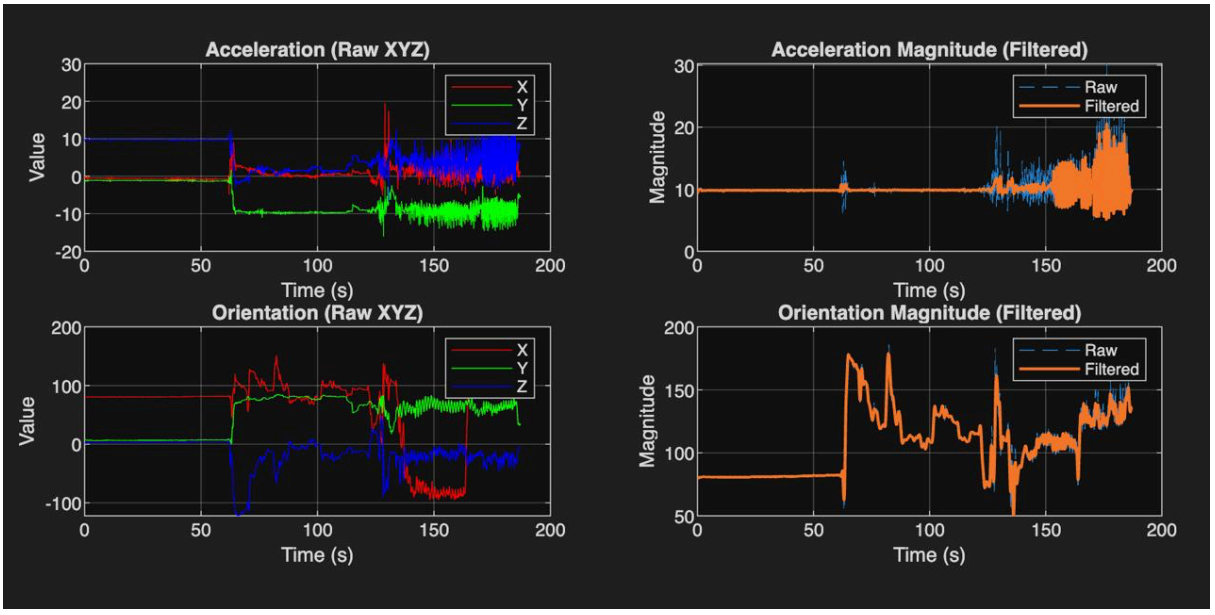
- After the Butterworth filter, the code applies `sgolayfilt` (polynomial order 3, frame length 11).
- This further smoothens the magnitude signal while preserving the important "peaks" that represent physical actions like steps, making the data much easier for the algorithm to interpret later.

3. Visual Analytics Dashboard:

The script generates a 4-plot dashboard to provide immediate visual feedback on the data quality:

- **Raw XYZ Plots**: Shows the raw tri-axial data to identify sensor bias or orientation shifts.

- **Comparison Plots**: Overlays the **Raw Magnitude** (dashed line) against the **Filtered Magnitude** (solid line). This allows you to verify that your filters are effectively removing noise without destroying the actual movement data.



Feautures laid →

Task 2→ Code 2 is the **Feature Extraction Engine** of our project. While Code 1 focuses on cleaning the data, Code 2 focuses on "quantifying" it—turning raw waves into specific numbers that a computer can use to distinguish between sitting, standing, and walking.

```
clc; clear; close all;

%% ================================
% STEP 1: LOAD DATA FILE
% ================================
filename = 'inss_3.mat';

if ~isfile(filename)
    error('Data file not found!');
end
```

```
data = load(filename);
vars = fieldnames(data);

disp("Feature extraction started...");

%% ================================
% STEP 2: FILTER SETTINGS
% ================================
fs = 50;          % Sampling frequency (Hz)
cutoff = 5;       % Low-pass cutoff frequency
window_sec = 60;  % Window size (seconds)

% Butterworth low-pass filter
[b,a] = butter(4, cutoff/(fs/2), 'low');

%% ================================
% STEP 3: PROCESS EACH SENSOR
% ================================
for i = 1:length(vars)

    varName = vars{i};
    varData = data.(varName);

    % Skip position data (not needed here)
    if contains(lower(varName), 'pos')
        continue;
    end

    % Work only with timetable data
    if istimetable(varData)

        tt = varData;

        % Convert time to seconds starting from 0
        time = seconds(tt.Properties.RowTimes - tt.Properti
es.RowTimes(1));
```

```
        % Check if data has X, Y, Z columns
        if width(tt) < 3
            continue;
        end

        fprintf('\nProcessing sensor: %s\n', varName);

        % Extract XYZ signals
        xyz = tt{:,1:3};

        %% ================================
        % STEP 4: SIGNAL PREPROCESSING
        % ================================

        % Compute magnitude (removes orientation dependenc
y)
        mag = sqrt(sum(xyz.^2, 2));

        % Apply filtering to remove noise
        mag_filtered = filtfilt(b, a, mag);       % Butterw
orth filter
        mag_filtered = sgolayfilt(mag_filtered,3,11); % Smo
othening

        %% ================================
        % STEP 5: WINDOW-BASED ANALYSIS
        % ================================
        total_time = time(end);
        num_windows = floor(total_time / window_sec);

        for w = 1:num_windows

            start_t = (w-1) * window_sec;
            end_t   = w * window_sec;

            % Extract window indices
            idx = (time >= start_t) & (time < end_t);
```

```
            if sum(idx) < 10
                continue;
            end

            % Segment data for current window
            seg_xyz = xyz(idx,:);
            seg_mag = mag_filtered(idx);

            %% ==============================
            % STEP 6: FEATURE EXTRACTION
            % ==============================

            % --- Basic statistics ---
            mean_xyz = mean(seg_xyz);
            var_xyz  = var(seg_xyz);

            mean_mag = mean(seg_mag);
            var_mag  = var(seg_mag);

            % --- RMS ---
            rms_xyz = sqrt(mean(seg_xyz.^2));
            rms_mag = sqrt(mean(seg_mag.^2));

            % --- Jerk (rate of change of acceleration) ---
            jerk = diff(seg_xyz) * fs;
            jerk_mag = sqrt(sum(jerk.^2,2));

            mean_jerk = mean(jerk_mag);
            var_jerk  = var(jerk_mag);

            % --- Step frequency (only for acceleration) --
-
            step_freq = NaN;

            if contains(lower(varName), 'acc')

                [peaks, ~] = findpeaks(seg_mag, ...
                    'MinPeakDistance', round(fs/2));
```

```
                duration = end_t - start_t;

                if duration > 0
                    step_freq = length(peaks) / duration;
                end
            end

            %% ================================
            % STEP 7: DISPLAY RESULTS
            % ================================
            fprintf('\nWindow %d (%d - %d sec)\n', w, start
_t, end_t);

            fprintf('Mean (X Y Z): %.3f  %.3f  %.3f\n', mea
n_xyz);
            fprintf('Variance (X Y Z): %.3f  %.3f  %.3f\n',
var_xyz);

            fprintf('RMS (X Y Z): %.3f  %.3f  %.3f\n', rms_
xyz);

            fprintf('Magnitude → Mean: %.3f | Var: %.3f | R
MS: %.3f\n', ...
                    mean_mag, var_mag, rms_mag);

            fprintf('Jerk → Mean: %.3f | Var: %.3f\n', ...
                    mean_jerk, var_jerk);

            if ~isnan(step_freq)
                fprintf('Step Frequency: %.3f steps/sec\n',
step_freq);
            end
        end
    end
end

disp("Feature extraction completed successfully!");
```

1. Segmentation via Windowing

Instead of analyzing the entire recording at once, the code breaks the data into **60-second windows**.

- **Purpose**: Human activity changes over time. By using windows, the algorithm can detect if you were sitting for the first minute and started walking in the second.
- **Logic**: It calculates the number of windows based on the total duration and loops through each segment to extract specific "fingerprints" of motion.

2. Statistical Features (The "Fingerprints")

The script calculates several mathematical features for every window to characterize the motion:

• **Mean (Average)**:

◦ Represents the central value of the signal.

◦ **Usage**: In acceleration, a mean magnitude near 9.8 m/s^2 indicates a static state (gravity), while higher values suggest vertical movement.

• **Variance**:

◦ Measures how much the signal spreads out from the mean.

◦ **Usage**: This is your best indicator of intensity. Sitting has near-zero variance (0.001), while walking shows significantly higher variance (4.72) due to the constant "bounce" of steps.

• **RMS (Root Mean Square)**:

◦ Calculates the square root of the arithmetic mean of the squares of the values.

◦ **Usage**: It represents the overall "power" or energy of the movement within that minute.

3. Advanced Motion Descriptors

Beyond basic statistics, the code calculates specialized IIoT features:

• **Jerk (Rate of Change of Acceleration)**:

◦ Calculated as the derivative of the acceleration signal multiplied by the sampling frequency (fs).

◦ **Usage**: High Jerk values indicate sudden, sharp movements or transitions (like standing up quickly), whereas low Jerk indicates smooth, steady motion.

• **Step Frequency (Peak Detection)**:

◦ The code uses the `findpeaks` function with a `MinPeakDistance` of half a second (fs/2) to identify individual footsteps.

◦ **Usage**: It divides the number of detected peaks by the window duration to get "steps per second

though we calculated the Advanced motion descriptors like jerk and Step frequency ,we have realised that calculating the mean ,variance and RMS velocity would serve our problem in distingusing the parameters.

Parameters worked on →

Code 3 is the **Intelligence and Decision Layer** of our system. While the previous codes focused on cleaning and measuring data, this script uses that data to perform **Activity Recognition** and **Scoring** through a weighted heuristic model.

```
clc; clear; close all;

%% ================================
% STEP 1: LOAD DATA
% ================================
filename = 'inss_3.mat';

if ~isfile(filename)
    error('Data file not found!');
end

data = load(filename);
vars = fieldnames(data);

disp("Starting feature extraction and scoring...");

%% ================================
% STEP 2: SETTINGS
% ================================
fs = 50;          % Sampling frequency (Hz)
cutoff = 5;       % Low-pass filter cutoff
window_sec = 60;  % Window duration (seconds)
```

```
% Design Butterworth low-pass filter
[b,a] = butter(4, cutoff/(fs/2), 'low');

%% ================================
% STEP 3: EXTRACT ORIENTATION INFO
% ================================
orientationY = [];
orientation_full = {};

for i = 1:length(vars)

    varName = vars{i};
    varData = data.(varName);

    % Process only orientation data
    if contains(lower(varName), 'ori') && istimetable(varDa
ta)

        tt = varData;

        % Convert time to seconds starting from zero
        time = seconds(tt.Properties.RowTimes - tt.Properti
es.RowTimes(1));

        ori_xyz = tt{:,1:3};

        total_time = time(end);
        num_windows = floor(total_time / window_sec);

        for w = 1:num_windows

            start_t = (w-1)*window_sec;
            end_t   = w*window_sec;

            idx = (time >= start_t) & (time < end_t);

            if sum(idx) < 10
                continue;
```

```
            end

            seg = ori_xyz(idx,:);

            % Mean orientation (used for posture detection)
            mean_ori = mean(seg);

            orientationY(w) = mean_ori(2);     % Y-axis (ti
lt)
            orientation_full{w} = seg(:,1);    % X-axis (fo
r direction change)
        end
    end
end

%% ==============================
% STEP 4: ACCELERATION + SCORING
% ==============================
total_CAS = 0;   % Cumulative Activity Score

for i = 1:length(vars)

    varName = vars{i};
    varData = data.(varName);

    % Process only acceleration data
    if contains(lower(varName), 'acc') && istimetable(varDa
ta)

        tt = varData;

        time = seconds(tt.Properties.RowTimes - tt.Properti
es.RowTimes(1));
        acc_xyz = tt{:,1:3};

        % Convert to magnitude (orientation independent)
        accMag = sqrt(sum(acc_xyz.^2,2));
```

```
        % Apply filtering to remove noise
        accMag = filtfilt(b,a,accMag);
        accMag = sgolayfilt(accMag,3,11);

        total_time = time(end);
        num_windows = floor(total_time / window_sec);

        fprintf('\nProcessing sensor: %s\n', varName);

        for w = 1:num_windows

            start_t = (w-1)*window_sec;
            end_t   = w*window_sec;

            idx = (time >= start_t) & (time < end_t);

            if sum(idx) < 10 || length(orientationY) < w
                continue;
            end

            seg_mag = accMag(idx);
            oriY = orientationY(w);

            %% ================================
            % STEP 5: COMPUTE SCORES
            % ================================

            % 1. Mobility → how much movement varies
            mobilityScore = var(seg_mag);

            % 2. Intensity → energy of movement
            intensity = mean(seg_mag.^2);

            % 3. Direction Change → turning behavior
            ori_signal = orientation_full{w};
            directionChange = sum(abs(diff(ori_signal)));

            %% ================================
```

```
            % STEP 6: NORMALIZATION
            % ================================
            mob_n = mobilityScore / (mobilityScore + 1);
            int_n = intensity / (intensity + 50);
            dir_n = directionChange / (directionChange + 10
0);

            %% ================================
            % STEP 7: COMBINED ACTIVITY SCORE
            % ================================
            w1 = 0.4;   % mobility weight
            w2 = 0.4;   % intensity weight
            w3 = 0.2;   % direction weight

            CAS = w1*mob_n + w2*int_n + w3*dir_n;

            total_CAS = total_CAS + CAS;

            %% ================================
            % STEP 8: ACTIVITY CLASSIFICATION
            % ================================
            activity = "Unknown";

            mean_mag = mean(seg_mag);

            if mean_mag < 10 && oriY < 15
                activity = "SITTING";

            elseif mean_mag < 10 && oriY > 45
                activity = "STANDING";

            elseif mean_mag > 10 && mean_mag < 11
                activity = "WALKING";

            elseif mean_mag > 11
                activity = "RUNNING";
            end
```

```
            %% ==============================
            % STEP 9: DISPLAY RESULTS
            % ==============================
            fprintf('\nWindow %d (%d - %d sec)\n', w, start
_t, end_t);

            fprintf('Mobility Score      : %.3f\n', mobilit
yScore);
            fprintf('Intensity Score     : %.3f\n', intensi
ty);
            fprintf('Direction Change    : %.3f\n', directi
onChange);

            fprintf('Combined Score (CAS): %.3f\n', CAS);
            fprintf('Detected Activity   : %s\n', activit
y);
        end
    end
end

fprintf('\nTotal Activity Score (CAS): %.3f\n', total_CAS);

disp("Feature extraction and classification completed.");
```

1. Dual-Sensor Fusion (Orientation + Acceleration)

Unlike the previous scripts, Code 3 synchronizes data from two different sensors to improve accuracy:

- **Orientation (The Posture Detector):** The script specifically extracts the **Y-axis tilt** (`oriY`).
  - **Logic:** If the phone is flat (low tilt), the user is likely sitting. If the phone is vertical (high tilt), the user is likely standing.
- **Acceleration (The Motion Detector):** It processes the filtered acceleration magnitude to determine the energy of the movement.

2. The Activity Scoring Model (CAS)

The script introduces a **Cumulative Activity Score (CAS)**, which is a multi-dimensional metric used to quantify physical exertion. It calculates three distinct components:

- **Mobility Score:** Derived from the **Variance** of the acceleration magnitude. It measures how much the movement fluctuates.
- **Intensity Score:** Derived from the **Mean Square** of the signal. it measures the raw "power" or force behind the movement.
- **Direction Change:** Calculated by summing the absolute differences in the orientation signal (`diff(ori_signal)`). This identifies if the user is walking in a straight line or turning frequently.

3. Normalization and Weighting

To ensure no single feature dominates the final score, the code applies a **Normalization** step using a saturation function `[x/(x+k)]`. It then combines them using a **Weighted Sum**:

• `Mobility (40%) + Intensity (40%) + Direction (20%) .`

• This creates a final score between 0 and 1 per window, which is then summed to get the **Total Activity Score**.

4. Heuristic Classification Logic

The "Intelligence" of the system comes from a set of threshold-based rules that classify the activity for each 60-second window:

Detected Activity	Acceleration Magnitude	Orientation (Y-Tilt)
SITTING	Low (<10)	Low (<15 degrees)
STANDING	Low (<10)	High (>45 degrees)
WALKING	Medium (10 to 11)	(Not weighted)
RUNNING	High (>11)	(Not weighted)

Algorithm Detailing →

1. Mobility Score

This parameter measures the dynamic variation or "bouncing" in the movement signal.

• **Formula:** $Mobility = \sigma^2(Mag_{filtered})$

• **Description:** It is the **Variance** of the filtered acceleration magnitude within a 60-second window.

• **Interpretation:** A higher variance indicates more erratic or high-energy

movement like running, while a value near zero indicates a static state like sitting.

2. Activity Intensity Score

This measures the total energy or power of the movement.

• **Formula:** $Intensity = \frac{1}{N}\sum_{i=1}^{N}(Mag_{filtered,i})^2$

• **Description:** It is the **Mean Square** (average of the squared values) of the filtered acceleration magnitude.

• **Interpretation:** It captures the "force" of the activity. Even if movement is steady, high intensity indicates a high-effort state.

3. Direction Change Score

This tracks how much the user is rotating or changing their heading.

• **Formula:** $DirChange = \sum |\Delta Orientation_X|$

• **Description:** It is the sum of the absolute differences of the orientation X-axis signal (`diff(ori_signal)`).

• **Interpretation:** High scores suggest the user is turning frequently or navigating corners, while low scores suggest moving in a straight line or staying still.

4. Cumulative Activity Score (CAS)

This is the final intelligence metric that combines all sensor data into a single representative value.

A. Normalization Step→

Before weighting, each score is normalized to a range of [0, 1] using a saturation function to prevent outliers from skewing the results:

• **Normalized Mobility** $mob_n = \frac{Mobility}{Mobility+1}$

• **Normalized Intensity** $int_n = \frac{Intensity}{Intensity+50}$

• **Normalized Direction** $dir_n = \frac{Direction}{Direction+100}$

B. Weighting Formula →

The CAS is calculated using the following weighted sum:

$$CAS = (w_1 \cdot mob_n) + (w_2 \cdot int_n) + (w_3 \cdot dir_n)$$

Parameter	Weight Variable	Value	Contribution
Mobility	w1	**0.4**	40%
Intensity	w2	**0.4**	40%
Direction	w3	**0.2**	20%

Final Outputs →

```
Starting feature extraction and scoring...

Processing sensor: Acceleration

Window 1 (0 - 60 sec)
Mobility Score      : 0.001
Intensity Score     : 96.166
Direction Change    : 7.980
Combined Score (CAS): 0.279
Detected Activity   : SITTING

Window 2 (60 - 120 sec)
Mobility Score      : 0.019
Intensity Score     : 97.468
Direction Change    : 869.786
Combined Score (CAS): 0.451
Detected Activity   : STANDING

Window 3 (120 - 180 sec)
Mobility Score      : 4.720
Intensity Score     : 113.760
Direction Change    : 2659.700
Combined Score (CAS): 0.801
Detected Activity   : WALKING

Total Activity Score (CAS): 1.531
Feature extraction and classification completed.
>> ✨ Press ⇧⌘P to generate code with Copilot
```

```
Window 1 (0 - 60 sec)
Mean (X Y Z): -0.672  -1.175  9.712
Variance (X Y Z): 0.022  0.007  0.004
RMS (X Y Z): 0.688  1.178  9.712
Magnitude → Mean: 9.806 | Var: 0.001 | RMS: 9.806
Jerk → Mean: 6.155 | Var: 67.460
Step Frequency: 0.250 steps/sec

Window 2 (60 - 120 sec)
Mean (X Y Z): 0.846  -9.035  2.058
Variance (X Y Z): 1.553  3.971  5.553
RMS (X Y Z): 1.505  9.252  3.128
Magnitude → Mean: 9.872 | Var: 0.019 | RMS: 9.873
Jerk → Mean: 27.223 | Var: 1785.088
Step Frequency: 0.267 steps/sec

Window 3 (120 - 180 sec)
Mean (X Y Z): 1.590  -8.978  3.835
Variance (X Y Z): 5.067  3.383  9.550
RMS (X Y Z): 2.755  9.164  4.925
Magnitude → Mean: 10.442 | Var: 4.720 | RMS: 10.666
Jerk → Mean: 100.995 | Var: 9096.365
Step Frequency: 0.800 steps/sec

Processing sensor: Orientation

Window 1 (0 - 60 sec)
Mean (X Y Z): 80.810  6.869  3.968
Variance (X Y Z): 0.202  0.145  0.361
RMS (X Y Z): 80.811  6.880  4.014
Magnitude → Mean: 81.201 | Var: 0.245 | RMS: 81.203
Jerk → Mean: 3.167 | Var: 68.021

Window 2 (60 - 120 sec)
Mean (X Y Z): 93.372  72.694  -27.875
Variance (X Y Z): 252.120  313.698  1184.564
RMS (X Y Z): 94.710  74.817  44.267
Magnitude → Mean: 126.471 | Var: 526.575 | RMS: 128.533
Jerk → Mean: 116.169 | Var: 27582.861

Window 3 (120 - 180 sec)
Mean (X Y Z): 14.029  62.138  -18.990
Variance (X Y Z): 7548.640  121.620  338.868
RMS (X Y Z): 87.937  63.107  26.437
Magnitude → Mean: 109.625 | Var: 354.808 | RMS: 111.229
Jerk → Mean: 359.079 | Var: 168885.020
Feature extraction completed successfully!
```